

بسم الله الرحمن الرحيم

King Abdulaziz University  
Faculty of Computing and information Technology  
Computer Science Department



# CPCS-214 Computer Architecture and Organization

## 26 - Division

# Introduction

- Reciprocal operation of multiply is divide
- An operation that is even less frequent and even more quirky
- It even offers opportunity to perform a mathematically invalid operation
  - dividing by 0

# Division Components

- **Dividend** number being divided
- **Divisor** number that the dividend is divided by
- **Quotient** primary result of a division
- **Remainder** secondary result of a division
- Another way to express relationship between components:

$$\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder}$$

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

|                             |                                   |          |
|-----------------------------|-----------------------------------|----------|
|                             |                                   | Quotient |
| Divisor $1000_{\text{ten}}$ | $\overline{1001010_{\text{ten}}}$ | Dividend |

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

$$\begin{array}{r} 1 \qquad \text{Quotient} \\ \text{Divisor } 1000_{\text{ten}} \overline{) 1001010_{\text{ten}}} \qquad \text{Dividend} \end{array}$$

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

|                             |                                    |          |
|-----------------------------|------------------------------------|----------|
|                             | 1                                  | Quotient |
| Divisor $1000_{\text{ten}}$ | $\overline{)1001010_{\text{ten}}}$ | Dividend |
|                             | <u>-1000</u>                       |          |

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

$$\begin{array}{r} \text{Quotient} \\ 1 \\ \text{Divisor } 1000_{\text{ten}} \overline{) 1001010_{\text{ten}}} \\ \underline{-1000} \\ 1 \end{array}$$

Dividend

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

$$\begin{array}{r} \text{Quotient} \\ 1 \\ \text{Divisor } 1000_{\text{ten}} \overline{) 1001010_{\text{ten}}} \\ \underline{-1000} \\ 10 \end{array}$$

Dividend



# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

$$\begin{array}{r} 10 \\ \text{Divisor } 1000_{\text{ten}} \overline{) 1001010_{\text{ten}}} \\ \underline{-1000} \\ 10 \end{array}$$

Quotient

Dividend

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

$$\begin{array}{r} 10 \\ \text{Divisor } 1000_{\text{ten}} \overline{) 1001010_{\text{ten}}} \\ \underline{-1000} \\ 10 \end{array}$$

Quotient

Dividend

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

|                             |                                    |          |
|-----------------------------|------------------------------------|----------|
|                             | 100                                | Quotient |
| Divisor $1000_{\text{ten}}$ | $\overline{)1001010_{\text{ten}}}$ | Dividend |
|                             | $\underline{-1000}$                |          |
|                             | 10                                 |          |
|                             | 101                                |          |

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

|                             |                                    |          |
|-----------------------------|------------------------------------|----------|
|                             | 1001.                              | Quotient |
| Divisor $1000_{\text{ten}}$ | $\overline{)1001010_{\text{ten}}}$ | Dividend |
|                             | $\underline{-1000}$                |          |
|                             | 10                                 |          |
|                             | 101                                |          |
|                             | 1010                               |          |

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

|                             |                                    |          |
|-----------------------------|------------------------------------|----------|
|                             | 1001.                              | Quotient |
| Divisor $1000_{\text{ten}}$ | $\overline{)1001010_{\text{ten}}}$ | Dividend |
|                             | $\underline{-1000}$                |          |
|                             | 10                                 |          |
|                             | 101                                |          |
|                             | 1010                               |          |
|                             | $\underline{-1000}$                |          |

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

|                             |                                    |          |
|-----------------------------|------------------------------------|----------|
|                             | $1001_{\text{ten}}$                | Quotient |
| Divisor $1000_{\text{ten}}$ | $\overline{)1001010_{\text{ten}}}$ | Dividend |
|                             | $\underline{-1000}$                |          |
|                             | 10                                 |          |
|                             | 101                                |          |
|                             | 1010                               |          |
|                             | $\underline{-1000}$                |          |
|                             | 10                                 |          |

# Decimal Division

- Start with example of long division using decimal numbers to recall names of operands and division algorithm
  - Divide  $1,001,010_{\text{ten}}$  by  $1000_{\text{ten}}$ :

|                             |                                    |           |
|-----------------------------|------------------------------------|-----------|
|                             | $1001_{\text{ten}}$                | Quotient  |
| Divisor $1000_{\text{ten}}$ | $\overline{)1001010_{\text{ten}}}$ | Dividend  |
|                             | $\underline{-1000}$                |           |
|                             | 10                                 |           |
|                             | 101                                |           |
|                             | 1010                               |           |
|                             | $\underline{-1000}$                |           |
|                             | $10_{\text{ten}}$                  | Remainder |

# Binary Division

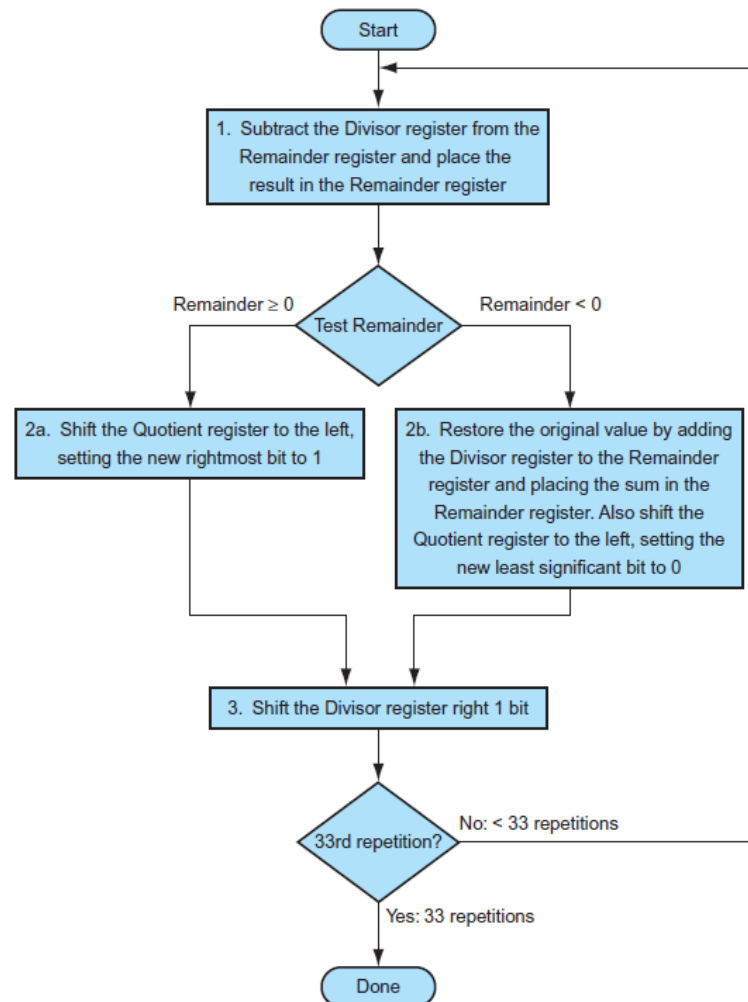
- Division algorithm tries to see how big a number can be subtracted, creating a digit of quotient on each attempt
- Decimal example uses only numbers 0 and 1, so it's easy to figure out how many times the divisor goes into the portion of dividend
  - It's either 0 times or 1 time
- Binary numbers contain only 0 or 1, so binary division is restricted to these two choices



# Binary Division

- Assume that both **dividend** and **divisor** are positive and hence **quotient** and **remainder** are nonnegative
- Division operands and both results are 32-bit values, and we will ignore the sign for now

# Algorithm



# Algorithm

- Figure shows three steps of division algorithm
- Computer isn't smart enough to know in advance whether divisor is smaller than dividend
  - It must first subtract the divisor in step 1
    - If result is positive, divisor was smaller or equal to dividend, so generate a 1 in quotient (step 2a)
    - If result is negative, next step is to restore original value by adding divisor back to remainder and generate a 0 in quotient (step 2b)
  - Divisor is shifted right and then we iterate again
  - Remainder and quotient will be found in their namesake registers

# Speedup

- Algorithm can be refined to be faster and cheaper
- Speedup comes from shifting operands and quotient simultaneously with subtraction
- This refinement halves width of adder and registers by noticing where there are unused portions of registers and adders

# Example

- Using a 4-bit version of the algorithm, divide  $7_{\text{ten}}$  by  $2_{\text{ten}}$ , or  $0000\ 0111_{\text{two}}$  by  $0010_{\text{two}}$ .

# Example (cont.)

| Iteration | Step           | Quotient | Divisor   | Remainder |
|-----------|----------------|----------|-----------|-----------|
| 0         | Initial values | 0000     | 0010 0000 | 0000 0111 |
| 1         |                |          |           |           |
|           |                |          |           |           |
|           |                |          |           |           |
| 2         |                |          |           |           |
|           |                |          |           |           |
|           |                |          |           |           |
| 3         |                |          |           |           |
|           |                |          |           |           |
|           |                |          |           |           |
| 4         |                |          |           |           |
|           |                |          |           |           |
|           |                |          |           |           |
| 5         |                |          |           |           |
|           |                |          |           |           |
|           |                |          |           |           |

# Example (cont.)

| Iteration | Step               | Quotient | Divisor   | Remainder |
|-----------|--------------------|----------|-----------|-----------|
| 0         | Initial values     | 0000     | 0010 0000 | 0000 0111 |
| 1         | 1: Rem = Rem – Div | 0000     | 0010 0000 | ①110 0111 |
|           |                    |          |           |           |
|           |                    |          |           |           |
| 2         |                    |          |           |           |
|           |                    |          |           |           |
|           |                    |          |           |           |
| 3         |                    |          |           |           |
|           |                    |          |           |           |
|           |                    |          |           |           |
| 4         |                    |          |           |           |
|           |                    |          |           |           |
|           |                    |          |           |           |
| 5         |                    |          |           |           |
|           |                    |          |           |           |
|           |                    |          |           |           |

# Example (cont.)

| Iteration | Step                                          | Quotient | Divisor   | Remainder |
|-----------|-----------------------------------------------|----------|-----------|-----------|
| 0         | Initial values                                | 0000     | 0010 0000 | 0000 0111 |
| 1         | 1: Rem = Rem – Div                            | 0000     | 0010 0000 | ①110 0111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0010 0000 | 0000 0111 |
|           |                                               |          |           |           |
| 2         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |
| 3         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |
| 4         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |
| 5         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |



# Example (cont.)

| Iteration | Step                                          | Quotient | Divisor   | Remainder |
|-----------|-----------------------------------------------|----------|-----------|-----------|
| 0         | Initial values                                | 0000     | 0010 0000 | 0000 0111 |
| 1         | 1: Rem = Rem – Div                            | 0000     | 0010 0000 | ①110 0111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0010 0000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0001 0000 | 0000 0111 |
| 2         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |
| 3         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |
| 4         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |
| 5         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |

# Example (cont.)

| Iteration | Step                                          | Quotient | Divisor   | Remainder |
|-----------|-----------------------------------------------|----------|-----------|-----------|
| 0         | Initial values                                | 0000     | 0010 0000 | 0000 0111 |
| 1         | 1: Rem = Rem – Div                            | 0000     | 0010 0000 | ①110 0111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0010 0000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0001 0000 | 0000 0111 |
| 2         | 1: Rem = Rem – Div                            | 0000     | 0001 0000 | ①111 0111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0001 0000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0000 1000 | 0000 0111 |
| 3         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |
| 4         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |
| 5         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |

# Example (cont.)

| Iteration | Step                                          | Quotient | Divisor   | Remainder |
|-----------|-----------------------------------------------|----------|-----------|-----------|
| 0         | Initial values                                | 0000     | 0010 0000 | 0000 0111 |
| 1         | 1: Rem = Rem – Div                            | 0000     | 0010 0000 | ①110 0111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0010 0000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0001 0000 | 0000 0111 |
| 2         | 1: Rem = Rem – Div                            | 0000     | 0001 0000 | ①111 0111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0001 0000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0000 1000 | 0000 0111 |
| 3         | 1: Rem = Rem – Div                            | 0000     | 0000 1000 | ①111 1111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0000 1000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0000 0100 | 0000 0111 |
| 4         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |
| 5         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |

# Example (cont.)

| Iteration | Step                                          | Quotient | Divisor   | Remainder |
|-----------|-----------------------------------------------|----------|-----------|-----------|
| 0         | Initial values                                | 0000     | 0010 0000 | 0000 0111 |
| 1         | 1: Rem = Rem – Div                            | 0000     | 0010 0000 | ①110 0111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0010 0000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0001 0000 | 0000 0111 |
| 2         | 1: Rem = Rem – Div                            | 0000     | 0001 0000 | ①111 0111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0001 0000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0000 1000 | 0000 0111 |
| 3         | 1: Rem = Rem – Div                            | 0000     | 0000 1000 | ①111 1111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0000 1000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0000 0100 | 0000 0111 |
| 4         | 1: Rem = Rem – Div                            | 0000     | 0000 0100 | ①000 0011 |
|           | 2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1  | 0001     | 0000 0100 | 0000 0011 |
|           | 3: Shift Div right                            | 0001     | 0000 0010 | 0000 0011 |
| 5         |                                               |          |           |           |
|           |                                               |          |           |           |
|           |                                               |          |           |           |

# Example (cont.)

| Iteration | Step                                          | Quotient | Divisor   | Remainder |
|-----------|-----------------------------------------------|----------|-----------|-----------|
| 0         | Initial values                                | 0000     | 0010 0000 | 0000 0111 |
| 1         | 1: Rem = Rem – Div                            | 0000     | 0010 0000 | ①110 0111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0010 0000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0001 0000 | 0000 0111 |
| 2         | 1: Rem = Rem – Div                            | 0000     | 0001 0000 | ①111 0111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0001 0000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0000 1000 | 0000 0111 |
| 3         | 1: Rem = Rem – Div                            | 0000     | 0000 1000 | ①111 1111 |
|           | 2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0 | 0000     | 0000 1000 | 0000 0111 |
|           | 3: Shift Div right                            | 0000     | 0000 0100 | 0000 0111 |
| 4         | 1: Rem = Rem – Div                            | 0000     | 0000 0100 | ①000 0011 |
|           | 2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1  | 0001     | 0000 0100 | 0000 0011 |
|           | 3: Shift Div right                            | 0001     | 0000 0010 | 0000 0011 |
| 5         | 1: Rem = Rem – Div                            | 0001     | 0000 0010 | ①000 0001 |
|           | 2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1  | 0011     | 0000 0010 | 0000 0001 |
|           | 3: Shift Div right                            | 0011     | 0000 0001 | 0000 0001 |

## Example (cont.)

- Figure shows value of each register for each of the steps, with the quotient being  $3_{\text{ten}}$  and the remainder  $1_{\text{ten}}$
- Notice that test in step 2 of whether remainder is positive or negative simply tests whether sign bit of Remainder register is a 0 or 1
- Surprising requirement of this algorithm is that it takes  $n + 1$  steps to get proper quotient and remainder

# Signed Division

- Simplest solution is to remember the signs of the divisor and dividend
- Then negate quotient if signs disagree

# Divide in MIPS

- The only requirement is a 64-bit register that can shift left or right and a 32-bit ALU that adds or subtracts
- Hence, MIPS uses 32-bit *Hi* and 32-bit *Lo* registers for divide
  - *Hi* contains remainder
  - *Lo* contains quotient after divide instruction completes



# Divide Instructions

- To handle both signed integers and unsigned integers, MIPS has two instructions:

- *divide* (**div**)

- *divide unsigned* (**divu**)

**div** *rs*, *rt* # signed: in [*Hi*,*Lo*]

**divu** *rs*, *rt* # unsigned: in [*Hi*,*Lo*]

# Divide Instructions

- To fetch integer 32-bit **lo-product**, use
  - *move from lo* (**mflo**)
- To fetch integer 32-bit **hi-product**, use
  - *move from lo* (**mfhi**)

# Divide Instructions

- MIPS assembler allows divide instructions to specify three registers, generating the `mflo` or `mghi` instructions to place the desired result into a general-purpose register

# MIPS assembly language

| Category           | Instruction                      | Example             | Meaning                                     | Comments                             |
|--------------------|----------------------------------|---------------------|---------------------------------------------|--------------------------------------|
| Arithmetic         | add                              | add \$s1,\$s2,\$s3  | \$s1 = \$s2 + \$s3                          | Three operands; overflow detected    |
|                    | subtract                         | sub \$s1,\$s2,\$s3  | \$s1 = \$s2 - \$s3                          | Three operands; overflow detected    |
|                    | add immediate                    | addi \$s1,\$s2,100  | \$s1 = \$s2 + 100                           | + constant; overflow detected        |
|                    | add unsigned                     | addu \$s1,\$s2,\$s3 | \$s1 = \$s2 + \$s3                          | Three operands; overflow undetected  |
|                    | subtract unsigned                | subu \$s1,\$s2,\$s3 | \$s1 = \$s2 - \$s3                          | Three operands; overflow undetected  |
|                    | add immediate unsigned           | addiu \$s1,\$s2,100 | \$s1 = \$s2 + 100                           | + constant; overflow undetected      |
|                    | move from coprocessor register   | mfc0 \$s1,\$epc     | \$s1 = \$epc                                | Copy Exception PC + special regs     |
|                    | multiply                         | mult \$s2,\$s3      | Hi, Lo = \$s2 × \$s3                        | 64-bit signed product in Hi, Lo      |
|                    | multiply unsigned                | multu \$s2,\$s3     | Hi, Lo = \$s2 × \$s3                        | 64-bit unsigned product in Hi, Lo    |
|                    | divide                           | div \$s2,\$s3       | Lo = \$s2 / \$s3,<br>Hi = \$s2 mod \$s3     | Lo = quotient, Hi = remainder        |
| Data transfer      | divide unsigned                  | divu \$s2,\$s3      | Lo = \$s2 / \$s3,<br>Hi = \$s2 mod \$s3     | Unsigned quotient and remainder      |
|                    | move from Hi                     | mfhi \$s1           | \$s1 = Hi                                   | Used to get copy of Hi               |
|                    | move from Lo                     | mflo \$s1           | \$s1 = Lo                                   | Used to get copy of Lo               |
|                    | load word                        | lw \$s1,20(\$s2)    | \$s1 = Memory[\$s2 + 20]                    | Word from memory to register         |
|                    | store word                       | sw \$s1,20(\$s2)    | Memory[\$s2 + 20] = \$s1                    | Word from register to memory         |
|                    | load half unsigned               | lhu \$s1,20(\$s2)   | \$s1 = Memory[\$s2 + 20]                    | Halfword memory to register          |
|                    | store half                       | sh \$s1,20(\$s2)    | Memory[\$s2 + 20] = \$s1                    | Halfword register to memory          |
|                    | load byte unsigned               | lbu \$s1,20(\$s2)   | \$s1 = Memory[\$s2 + 20]                    | Byte from memory to register         |
|                    | store byte                       | sb \$s1,20(\$s2)    | Memory[\$s2 + 20] = \$s1                    | Byte from register to memory         |
|                    | load linked word                 | ll \$s1,20(\$s2)    | \$s1 = Memory[\$s2 + 20]                    | Load word as 1st half of atomic swap |
| Logical            | store conditional word           | sc \$s1,20(\$s2)    | Memory[\$s2+20]=\$s1;\$s1=0 or 1            | Store word as 2nd half atomic swap   |
|                    | load upper immediate             | lui \$s1,100        | \$s1 = 100 * 2 <sup>16</sup>                | Loads constant in upper 16 bits      |
|                    | AND                              | and \$s1,\$s2,\$s3  | \$s1 = \$s2 & \$s3                          | Three reg. operands; bit-by-bit AND  |
|                    | OR                               | or \$s1,\$s2,\$s3   | \$s1 = \$s2   \$s3                          | Three reg. operands; bit-by-bit OR   |
|                    | NOR                              | nor \$s1,\$s2,\$s3  | \$s1 = ~ (\$s2   \$s3)                      | Three reg. operands; bit-by-bit NOR  |
|                    | AND immediate                    | andi \$s1,\$s2,100  | \$s1 = \$s2 & 100                           | Bit-by-bit AND with constant         |
|                    | OR immediate                     | ori \$s1,\$s2,100   | \$s1 = \$s2   100                           | Bit-by-bit OR with constant          |
| Conditional branch | shift left logical               | sll \$s1,\$s2,10    | \$s1 = \$s2 << 10                           | Shift left by constant               |
|                    | shift right logical              | srl \$s1,\$s2,10    | \$s1 = \$s2 >> 10                           | Shift right by constant              |
|                    | branch on equal                  | beq \$s1,\$s2,25    | if (\$s1 == \$s2) go to PC + 4 + 100        | Equal test; PC-relative branch       |
|                    | branch on not equal              | bne \$s1,\$s2,25    | if (\$s1 != \$s2) go to PC + 4 + 100        | Not equal test; PC-relative          |
|                    | set on less than                 | slt \$s1,\$s2,\$s3  | if (\$s2 < \$s3) \$s1 = 1;<br>else \$s1 = 0 | Compare less than; two's complement  |
|                    | set less than immediate          | slti \$s1,\$s2,100  | if (\$s2 < 100) \$s1 = 1;<br>else \$s1 = 0  | Compare < constant; two's complement |
|                    | set less than unsigned           | sltu \$s1,\$s2,\$s3 | if (\$s2 < \$s3) \$s1 = 1;<br>else \$s1 = 0 | Compare less than; natural numbers   |
| Unconditional jump | set less than immediate unsigned | sltiu \$s1,\$s2,100 | if (\$s2 < 100) \$s1 = 1;<br>else \$s1 = 0  | Compare < constant; natural numbers  |
|                    | jump                             | j 2500              | go to 10000                                 | Jump to target address               |
|                    | jump register                    | jr \$ra             | go to \$ra                                  | For switch, procedure return         |
|                    | jump and link                    | jal 2500            | \$ra = PC + 4; go to 10000                  | For procedure call                   |